Спецификация модуля ККД (контроль качества данных по целям)

Проект: «Автотрекинг целей B2B+», модуль «Цели» в ПУЛЬСе, раздел «Автотрекер» в АС Поток. Дата: 08.11.2026.

1. Контекст и цель

Сотрудники заносят цели и ключевые результаты (КР) в модуле «Цели». Если данные заведены некорректно — цель без ответственного, без КР, веса не сходятся, цель не привязана к цели вышестоящего уровня — процесс целеполагания работает неэффективно, а руководство не видит реальной картины.

Сейчас качество данных проверяется, но только техническими метриками, без нужной визуализации. В этом документе описан модуль ККД: он еженедельно проверяет данные по целям и КР, находит проблемы, считает метрики и показывает их на отдельной странице вместе с дельтой к прошлому прогону.

Результат этой спецификации — вход для команды разработки: по ней пишется код и приёмочные тесты, без возвратов к аналитику за уточнениями.

2. Глоссарий

> В глоссарий включены только термины, введённые самой спецификацией — это принятые здесь решения, а не общеизвестные слова: без фиксированных определений они допускают разные трактовки и влияют на все правила, метрики и дельту.

Термин	Определение
ККД	Контроль качества данных — модуль, который проверяет данные по целям и КР, находит проблемы и показывает отчёт руководителям и администраторам.
Прогон	Одна единоразовая проверка всех активных целей по снимку данных. Проводится еженедельно; у каждого прогона есть дата (`snapshot_date`).
Активная цель	Цель, которая участвует в расчёте ККД и считается действующей на дату прогона. Точный критерий — см. раздел 3 «Ключевые решения».
Каскадирование	Связь цели с целью вышестоящего уровня (родительской) — по подразделению и по дереву целей.
Валидный каскад	Цель привязана к активной цели вышестоящего уровня *либо* является корневой целью подразделения верхнего уровня (`unit.is_root = TRUE`). Используется метрикой полноты каскада; граница нарушения — см. раздел 3.
Проблема	Нарушение одного из правил проверки, найденное в конкретной цели (или её КР) на конкретном прогоне.

Дельта	Изменение набора найденных проблем по сравнению с прошлым успешным прогоном: какие проблемы появились, какие исчезли.
Уровни важности	`critical` **Критический** — цель теряет смысл как объект управления (её нельзя анализировать). `major` **Существенный** — не ломает процесс, но искажает общую картину. `minor` **Незначительный** — гигиеническое замечание, не влияющее на расчёт.

3. Ключевые решения

> Спорные места зафиксированы здесь и на них ссылается остальной документ — это исключает неоднозначное прочтение.

Вопрос	Решение
Какая цель считается активной	Цель считается активной и попадает в проверки ККД, если выполняются два условия одновременно: 1) статус цели — `active`; 2) дата прогона попадает в период действия цели: `snapshot_date BETWEEN start_date AND due_date`. Все остальные цели (черновики, закрытые, отменённые, а также цели, у которых период ещё не начался или уже истёк) в расчёт не попадают и проблемами не считаются.
Единица учёта	Цель, а не КР: проблема КР считается проблемой её цели. Так метрики сходятся со списком проблем на странице: и там, и там считаем цели.
Двойной учёт проблем	Исключён: одна цель может нарушить несколько правил (например, и каскад, и веса). В метрике «доля корректных целей» цель считается один раз, даже если у неё несколько проблем. В дельте каждая проблема учитывается отдельно — по типам правил.
Веса КР, если не заведены	Сумма пустых весов принимается равной 0, что ≠ 100% — нарушение `KR_WEIGHTS_SUM`. Частично заполненные веса суммируются по заполненным.
Граница правила каскадирования	Нарушение — если активная цель не привязана к родительской цели (`parent_goal_id = NULL`) и не является корневой целью подразделения верхнего уровня (`unit.is_root = FALSE`). Корневая цель подразделения верхнего уровня валидна.
Пустое подразделение	Значение метрики — `null`, а не 0: отсутствие целей — это отсутствие данных, а не нулевое качество.
Округление	Доли округляются до одного знака после запятой.
Направление «лучше/хуже»	Вычисляет бэкенд и передаёт в поле `trend`; фронтенд не переопределяет его по знаку дельты.

status и delta_status	Разделены: `status` — жизненный цикл проблемы в текущем прогоне (`open` / `resolved`); `delta_status` — вычисляемый атрибут сравнения прогонов (`new` / `persisting` / `resolved`).
Справочник правил	Статический, сидируется миграцией (откатывается вместе со схемой). Изменение уровня важности правила — правка одной строки, не кода.

4. Модель данных

Модель данных ККД включает 7 сущностей: 3 описывают предметную область (подразделения, цели, КР), 1 — исполнителей (сотрудники), 3 — результаты работы модуля ККД (Прогоны, Правила, Проблемы).

Сущность «сотрудник» выделена по 2 причинам:

1. по условию цели заводятся сотрудниками — ответственный является полноправной сущностью, а не текстовым полем;
2. ФИО сотрудника нужно для отображения ответственного на странице ККД.

units — подразделения

Атрибут	Тип	Описание
id	UUID (PK)	Уникальный идентификатор подразделения
name	String	Название подразделения
parent_id	UUID (FK → units), NULL для верхнего уровня	Вышестоящее подразделение; образует иерархию
is_root	Boolean	Флаг верхнего уровня иерархии

employees — сотрудники

Атрибут	Тип	Описание
id	UUID (PK)	Уникальный идентификатор сотрудника
fio	String	ФИО сотрудника

goals — цели

Атрибут	Тип	Описание
id	UUID (PK)	Уникальный идентификатор цели
title	String	Название цели
unit_id	UUID (FK → units)	Подразделение, к которому относится цель

owner_id	UUID (FK → employees)	Ответственный за цель
parent_goal_id	UUID (FK → goals), NULL только для корневых	Родительская цель в каскаде
status	Enum: draft, active, closed, cancelled	Статус цели в жизненном цикле
start_date	Date, NOT NULL	Начало периода действия цели
due_date	Date, NOT NULL	Срок (конец периода) достижения цели
created_at	Timestamp	Дата создания записи

key_results — ключевые результаты

Атрибут	Тип	Описание
id	UUID (PK)	Уникальный идентификатор KP
goal_id	UUID (FK → goals)	Цель, к которой относится KP
title	String	Название KP
target_value	Numeric	Целевое значение KP
actual_value	Numeric, NULL	Фактическое значение KP
weight	Numeric	Вес KP в цели
fact_updated_at	Timestamp, NULL	Дата последнего обновления факта

kkd_runs — прогоны проверки

Атрибут	Тип	Описание
id	UUID (PK)	Уникальный идентификатор прогона
snapshot_date	Date	Дата снимка данных, на который выполнен расчёт
started_at	Timestamp	Момент запуска прогона
finished_at	Timestamp, NULL	Момент завершения прогона
status	Enum: running, completed, failed	Статус прогона
compared_run_id	UUID (FK → kkd_runs), NULL	Прогон, с которым выполнено сравнение (дельта)

kkd_rules — правила (справочник)

Атрибут	Тип	Описание
code	String (PK)	Код правила

severity	Enum: critical, major, minor	Уровень важности правила

kkd_issues — найденные проблемы

Атрибут	Тип	Описание
id	UUID (PK)	Уникальный идентификатор проблемы
run_id	UUID (FK → kkd_runs)	Прогон, в котором проблема найдена
rule_code	String (FK → kkd_rules)	Код правила, по которому найдена проблема
goal_id	UUID (FK → goals)	Цель, к которой относится проблема

> **Примечание о развитии (бэклог).** Модель сознательно не включает проверки, не предусмотренные текущим набором правил: незаполненное целевое значение КР, несвежий факт КР (требует параметра N дней), цель в статусе active с истёкшим сроком. Все три проверки добавляются в последующих версиях без изменения формул метрик — необходимые для них атрибуты (target_value, fact_updated_at) уже присутствуют в модели данных.

5. Правила проверки (4 правила)

Проверяются только активные цели (критерий — раздел 3). Правило фиксирует проблему у цели; если нарушение в КР — проблема относится к цели этого КР.

Код	Объект	Важность	Условие нарушения	Почему это проблема
`GOAL_NO_OWNER`	Цель	`critical`	У активной цели не заполнен ответственный: `goals.owner_id` IS NULL	Цель «ничья»: непонятно, кто отвечает за её достижение и исправление данных
`GOAL_NO_KR`	Цель	`critical`	У активной цели нет ни одной записи в `key_results`, где `key_results.goal_id` = `goals.id`	Цель не измерима: по ней нельзя оценить прогресс
`KR_WEIGHTS_SUM`	Цель (её КР)	`major`	Сумма весов КР цели не равна 100% ИЛИ у какого-то КР вес не заполнен (`key_results.weight` IS NULL)	Прогресс цели считается неверно — картина для руководителя искажена
`CASCADE_NO_PARENT`	Цель	`major`	Активная цель не привязана к родительской цели (`parent_goal_id = NULL`) и при этом не является корневой целью подразделения верхнего уровня (`unit.is_root = FALSE`)	Разрыв дерева целеполагания: цель «висит» вне структуры, вклад подразделения не прослеживается

> Проверка «дата обновления факта KP» в правила не включена сознательно: у новой цели факта ещё нет, и это нормальные данные. Осмысленная проверка возможна только с порогом «факт старше N дней», а параметра N в требованиях нет — проверка отнесена в бэклог (раздел 4).

6. Метрики качества данных

Все три метрики считаются по активным целям на дату прогона. Единица измерения у всех — проценты (%), «лучше» — рост. Учёт дочерних подразделений — п. 6.2; пустое подразделение, округление и направление «лучше/хуже» — раздел 3 «Ключевые решения».

№	Метрика	Ключ	Формула	Единица	«Лучше»
1	Доля корректных целей	`correct_goals_share`	Активные цели без нарушений уровня `critical` и `major` / все активные цели × 100	%	Рост
2	Доля целей с корректным каскадированием	`cascade_completeness`	Активные цели без нарушения `CASCADE_NO_PARENT` / все активные цели × 100	%	Рост
3	Доля целей с корректными весами KP	`kr_weights_share`	Активные цели, у которых все веса KP заполнены и сумма равна 100% / все активные цели × 100	%	Рост

6.1. Обоснование

Метрика 1. ККД нужен, чтобы данные о целях были полными и правильными: от этого зависит дальнейшая обработка и статистика. Поэтому «корректная» цель — без нарушений и `critical`, и `major` уровня. Незначительные (`minor`) не учитываем, чтобы мелкие замечания не дёргали метрику еженедельно.

Метрика 2. Прямая реализация требования задания: каскадирование упомянуто в условиях отдельно и не один раз. Отдельный срез нужен, потому что сломанный каскад в дочерних подразделениях не виден в общей метрике подразделения.

Метрика 3. Вес — единственный параметр KP, который входит в текущие правила. Цель без KP считается некорректной по определению (это предельный случай незаполненного блока KP) и уже учтена в метрике 1 как критическое нарушение.

Как связаны метрики. Метрика 1 — общий уровень, метрики 2 и 3 — детализация по направлениям. Каждая существенная проблема видна и в метрике 1, и в своём срезе. Это два уровня детализации, а не дублирование.

6.2. Как метрика подразделения учитывает дочерние подразделения

Метрика подразделения по умолчанию считается по дереву: цели самого подразделения плюс цели всех дочерних подразделений (рекурсивно по `units.parent_id`). Параметр запроса `include_children=false` включает расчёт только по целям самого подразделения — он нужен, чтобы видеть собственные данные отдельно от данных подчинённых. Значение по умолчанию — `true`.

7. Дельта с прошлым прогоном

Вопрос	Решение
Как понять, что проблема «та же самая»?	Ключ сравнения — цель + правило: считаем проблему той же, если в прошлый и текущий прогонах (оба успешные) найдено нарушение одного и того же правила у одной и той же цели. Проблемы на каждом прогоне создаются как новые записи с новыми `id`, поэтому сравнивать их по `id` нельзя — только по смыслу: цель + правило.
Первый прогон — сравнивать не с чем	В ответе API блок дельты возвращает `null`, а на странице показывается сообщение «первый прогон — сравнение будет доступно со следующего прогона». Метрики и список проблем при этом показываются как обычно.
Прошлый прогон завершился с ошибкой (status = failed)	Дельта считается с последним успешно завершённым прогоном. Если успешных прогонов нет вообще — применяется правило первого прогона. Сами метрики считаются от текущего прогона и не зависят от прошлого. Прогон, с которым сравнивали, фиксируется в `compared_run_id` (раздел 4) и возвращается в поле `compared_to`.

Категории дельты:

Категория	Условие
`new` (новая)	Пара «правило + цель» есть в текущем прогоне и отсутствует в предыдущем успешном
`persisting` (сохраняющаяся)	Пара есть и в текущем, и в предыдущем успешном прогоне
`resolved` (решённая)	Пара есть в предыдущем успешном прогоне и отсутствует в текущем

> Прогон со статусом `running` в сравнение не попадает никогда — только `completed`. Если текущий прогон `failed`, страница показывает последний успешный отчёт с сообщением «данные от <даты>».

8. Контракт API

Три метода отдают на страницу ККД статистику, список проблем и состояние прогона. Ответы — JSON в кодировке UTF-8. Базовый URL: `http://<host>:<port>/api`. Все методы работают с последним успешно завершённым прогоном, если не передан `run_id`. Аутентификация в рамках тестового задания не реализуется (см. раздел 4).

8.1. Методы и параметры

Метод и путь	Назначение	Параметры	Возвращает
GET `/api/v1/kkd/summary`	Сводка по прогону: метрики, счётчики проблем, дельта	`unit_id` — подразделение (необязателен: без него — вся компания); `include_children` — учитывать дочерние подразделения (необязателен, по умолчанию `true`); `run_id` — прогон (необязателен: по умолчанию последний успешный)	Метрики подразделения (с учётом дочерних) и дельта с прошлым успешным прогоном
GET `/api/v1/kkd/issues`	Список найденных проблем с фильтрами и постраничной выдачей	`unit_id` — подразделение (необязателен); `delta_status` — фильтр по дельта-статусу: `new` / `persisting` (необязателен). Значения `resolved` в фильтре нет: решённые проблемы в текущем прогоне отсутствуют, их видно только в счётчике `delta.resolved` сводки; `severity` — фильтр по важности: `critical` / `major` / `minor` (необязателен); `limit`, `offset` — пагинация (необязательны, по умолчанию limit = 50, максимум 200)	Список найденных проблем с фильтрацией и общим количеством
GET `/api/v1/kkd/runs/{id}`	Просмотр статуса и деталей прогона — в том числе чтобы узнать, с чем сравнивали	`id` — идентификатор прогона (обязателен, в пути)	Информация о прогоне: статус, даты, с каким прогоном сравнивали

8.2. Структура ответа по методам

Метод	Поле	Тип	Описание
GET /summary			
	`run_id`	UUID	Прогон, по которому посчитан ответ
	`compared_to`	UUID | null	Прогон, с которым сравнили; `null` для первого прогона
	`metrics[]`	массив	Метрики — по одной записи на каждую из трёх метрик
	`metrics[].key`	string	Ключ метрики: `correct_goals_share`, `cascade_completeness`, `kr_weights_share`
	`metrics[].name`	string	Название метрики для отображения

	`metrics[].value`	number | null	Значение в процентах, округлено до одного знака; `null` — в подразделении нет целей (см. раздел 3)
	`metrics[].unit`	string	Единица измерения: `percent`
	`metrics[].trend`	string | null	Направление «лучше/хуже» относительно прошлого прогона; считает бэкенд, фронтенд не пересчитывает
	`metrics[].delta`	number | null	Изменение в процентных пунктах относительно прошлого успешного прогона; `null` — сравнивать не с чем
	`issues`	объект	Счётчики проблем текущего прогона по уровням важности — для шапки отчёта
	`issues.critical` / `issues.major` / `issues.minor`	integer	Количество проблем соответствующего уровня в текущем прогоне
	`delta`	объект | null	Счётчики категорий дельты; `null` — первый прогон (раздел 7)
	`delta.new` / `delta.persisting` / `delta.resolved`	integer	Сколько проблем новых, сохраняющихся и решённых относительно предыдущего успешного прогона (категории — раздел 7)
GET /issues			
	`total`	integer	Общее количество проблем по фильтру (для пагинации)
	`items[]`	массив	Проблемы текущего прогона
	`items[]` : элементы	—	`id`, `rule_code`, `severity`, `object_type`, `object_id`, `goal_title`, `unit_id`, `status` (`open` / `resolved`), `fingerprint`, `found_at`
	`items[].delta_status`	string | null	Категория дельты: `new` / `persisting`. Вычисляется при сравнении с предыдущим успешным прогоном, в БД не хранится (см. раздел 4); `null` — первый прогон. Используется для меток на странице и фильтра из 8.1. Решённые проблемы в текущем прогоне отсутствуют, поэтому значения `resolved` у элементов списка нет — они учтены счётчиком `delta.resolved`
GET /runs/{id}			
	`id`	UUID	Идентификатор прогона
	`status`	string	`running` / `completed` / `failed`

	`snapshot_date`	date	Дата, на которую зафиксированы данные прогона
	`started_at`	timestamp	Момент запуска
	`finished_at`	timestamp | null	Момент завершения; `null`, пока прогон не завершился
	`compared_to`	UUID | null	С каким прогоном сравнивали

8.3. Коды ошибок (общие для всех методов)

Код	Когда возвращается	Сообщение
400	Некорректный параметр: неизвестный `delta_status` или `severity`, отрицательный или превышающий максимум (200) `limit`, невалидный UUID	«Некорректное значение параметра {param_name}»
404	Подразделение или прогон с указанным `id` не найдены	«Подразделение не найдено» / «Прогон не найден»
409	Прогон ещё выполняется (`status = running`) — данные не финальны, страница не должна показывать их как результат проверки	«Прогон ещё выполняется, данные не финальны. Повторите запрос после его завершения»
500	Внутренняя ошибка при расчёте метрик или выборке данных	«Внутренняя ошибка расчёта ККД. Повторите запрос позже»

> {param_name} — шаблон: при ответе подставляется имя параметра (например, «Некорректное значение параметра severity»). Один шаблон на код 400 вместо отдельного текста на каждый параметр.

8.4. Пример ответа целиком — GET /api/v1/kkd/summary

Контрольные числа примера: 100 активных целей, из них нарушений critical — 2, major — 21 (9 по каскадированию, 15 по весам, 3 цели с обоими нарушениями), итого проблемных 23 цели → корректных 77 целей → метрика 1 = 77%.

```
{
  "run_id": "3f1a2b4c-0000-4a10-9c11-2e6f8a71b001",
  "compared_to": "3f1a2b4c-0000-4a10-9c11-2e6f8a71b000",
  "metrics": [
    {
      "key": "correct_goals_share",
      "name": "Доля корректных целей",
      "value": 77.0,
      "unit": "percent",
      "trend": "worse",
      "delta": -4.0
    },
    {
      "key": "cascade_completeness",
      "name": "Доля целей с корректным каскадированием",
```

```
      "value": 91.0,
      "unit": "percent",
      "trend": "worse",
      "delta": -2.0
    },
    {
      "key": "kr_weights_share",
      "name": "Доля целей с корректными весами КР",
      "value": 85.0,
      "unit": "percent",
      "trend": "better",
      "delta": 3.0
    }
  ],
  "issues": {
    "critical": 2,
    "major": 24,
    "minor": 0
  },
  "delta": {
    "new": 8,
    "persisting": 18,
    "resolved": 5
  }
}
```

> Сходимость примера с разделами 6–7: 100 − 23 проблемных цели = 77% — сходится с формулой метрики 1 (23 = 2 цели critical + 21 цель major, где 21 = 9 каскадирование + 15 веса − 3 цели с обоими нарушениями). Дельта: 8 новых + 18 сохраняющихся = 26 открытых проблем — по числу нарушений: 2 critical + 9 каскадирование + 15 веса (три цели нарушили по два major-правила, каждая даёт отдельную проблему, см. раздел 3). Блок `issues` сходится с дельтой: critical 2 + major 24 (9 + 15) + minor 0 = 26 = 8 новых + 18 сохраняющихся. Для первой метрики дельта = −4,0 п.п.: рост был бы «лучше», поэтому `trend: "worse"`; для третьей дельта = +3,0 п.п. при росте — `trend: "better"` (направление считает бэкенд).

8.5. Поведение в граничных случаях

Случай	Поведение
Подразделение без целей	Метрики возвращаются со значением `null`, а не 0 — отсутствие целей это отсутствие данных, а не нулевое качество (см. раздел 3). Дельта для такой метрики — `null`
Цель без ключевых результатов	Это нарушение critical GOAL_NO_KR: цель попадает в список проблем метода `/issues` и в числитель/знаменатель метрик как некорректная. API при этом отдаёт обычный ответ — граничный случай не влияет на контракт
Прогон ещё выполняется	`409 Conflict`. Частичные данные не отдаём: ответ был бы воспринят как результат проверки. На странице — сообщение «прогон выполняется, данные будут доступны после завершения». Статус прогона можно посмотреть через `GET /runs/{id}`
Первый прогон	`compared_to = null`, блок дельты — `null`, метрики — обычные значения (см. раздел 7)
Пустой результат фильтрации	Не ошибка: `200 OK`, `total = 0`, `items = []` — например, новых проблем по фильтру нет. На странице вместо списка показывается пустое состояние

Последний прогон завершился с ошибкой	Все методы по умолчанию работают с последним **успешно завершённым** прогоном; прогон со статусом `failed` источником данных не является. Страница показывает отчёт по последнему успешному прогону с сообщением «прогон от <даты> завершился с ошибкой — показаны данные от <даты>»: первая дата — упавший прогон, вторая — последний успешный. Дельта считается между двумя последними успешными прогонами (раздел 7)

9. Трассировка: пункты задания и где они закрыты

Требование задания	Где в документе
Модель: подразделения с иерархией, цели (ответственный, родительская цель, статус, срок), КР (целевое и фактическое значение, вес, дата обновления факта), прогоны и проблемы	Раздел 4
4 правила, минимум одно на каскадирование; код, объект, уровень, условие текстом	Раздел 5
Что считается активной целью	Раздел 3 (таблица «Ключевые решения»), глоссарий (раздел 2)
3 метрики: формула, единица измерения, «лучше» — рост/снижение, учёт дочерних подразделений	Разделы 6 и 6.2
Дельта: «та же проблема», первый прогон, ошибка прошлого прогона	Раздел 7
Контракт API: 2–3 метода, параметры, структура ответа, пример целиком, граничные случаи	Раздел 8
Еженедельная периодичность проверок	Глоссарий (прогон), раздел 1
Однозначность: реализация без возвратов к аналитику	Проверяемый критерий активной цели (раздел 3), контроль сходимости чисел (8.4), граничные случаи (8.5), трассировка

10. Мотивация участия

Почему интересен проект

Для меня это возможность применить знания на реальном продукте: до сих пор я работала с учебными задачами, и мне важно пройти полный цикл системного аналитика на боевом проекте — от модели данных и правил проверки до контракта API и приёмочных тестов. Сама задача тоже откликается: ККД — это не абстрактная техническая метрика, а инструмент, от которого зависит, видит ли руководство реальную картину по целям. Интересно сделать так, чтобы руководитель получал не «сырые цифры», а понятный ответ: что не так с целями, где разрыв в каскаде, что изменилось с прошлой недели.

Роль в команде

Вижу себя связующим звеном между бизнес-заказчиком и разработкой: перевожу требования с языка бизнеса на язык точных, проверяемых правил — так, чтобы разработчик писал код и приёмочные тесты без возвратов за уточнениями. В этом проекте это значит: формализовать правила, метрики и контракт API так, чтобы не оставалось двусмысленности, и поддерживать спецификацию в актуальном состоянии по ходу разработки.

Время

Готова уделять проекту 20 часов в неделю в течение 3 месяцев.